

JS topic test – Scope of variable , hoisting

Part1: Variables

- What is a variable?
A container that stores data.
Eg : 10 stores at variable x.
i.e. `var x =10 ;`
- Why do we need variables?
How we show that the variable is stored in memory at this place so we use variables to label the memory place.
- Difference between `var`, `let`, and `const`.

parameters	Let	var	const
redeclared	no	yes	no
updated	yes	yes	no
hoisted	TDZ(initialization)	Yes but as undefined	TDZ
scope	block	function	block

- Which keyword was introduced in ES6?
Let and Const
- Which keyword should we use most of the time?
Depending on the situation, we use let when we want to change the values and if we want const values then we use const.
- Why is `var` considered problematic?
There are many faults in this :
 1. As it is function scoped, this creates variable i at once and updates on every loop which creates a bug within loop and setTimeout.
 2. We can redeclare a variable using var which also creates a bug in a large codebase.

This creates a bug in loop and async functions.
- Can we declare variables without a keyword?
Not

- What happens if we do?
That doesn't do anything; it gives blank output.
- Difference between declaration and initialization.
Declaration = let x ;
Initialization = x = 10; assign value to the variable.
- What is an assignment?

Any value assigned to the variable called assignment.

x=10;

Part 2: Var

- Is **var** function-scoped or block-scoped?

Function-scoped
- Can **var** be redeclared?

yes
- Can **var** be reassigned?

Yes
- Is **var** hoisted?

Yes with undefined value
- What value does it get before initialization?

undefined
- Does **var** create a property on the global object?

Yes

Part 3: Let

- Is **let** block-scoped?

Yes

- Can `let` be redeclared?

no

- Can `let` be reassigned?

yes

- Is `let` hoisted?

TDZ issue

- What is the Temporal Dead Zone?

The zone between the declaration and initialization is called Temporal Dead Zone.

- Why does TDZ exist?

Because variables have to wait for initialization after once it goes to memory that time before initialization called TDZ, as it is dead before we assign any value.

- Can we access a `let` variable before declaration?

No

Part 4: Const

- Can `const` be reassigned?

not

- Can `const` be redeclared?

Not

- Why must `const` be initialized during declaration?

Because const needs to have constant value so once we have to assign any value in it.

- Does `const` make an object immutable?

No, because we can change the value of property of an object but we cannot change the whole object at once because it is constant.

```
const obj = {a:10, b:20};
```

```
obj.a=15; //obj = {a:15,b:20}
```

But if we try to write like:

```
obj= {a:15, b:20};
```

Then , it creates an error.

- Difference between immutable binding and immutable objects.

Part 5: Scope

- What is scope?

Scope is the area or boundary of the variables where we can access it.

- Types of scope in JavaScript.

Global,function,block scope

- What is Global Scope?

Global scope is the scope where we can access variables anywhere in the codebase.

- What is Function Scope?

Function scope where we can access variables in the function.

- What is Block Scope?

Block scope where we can access variables in the block with {}.

- Can **var** access a block variable?

not

- Can **let** access outside its block?

not

- What is Lexical Scope?

Lexical scope is the scope of variables from the outer environment of the Variables.

- How does JavaScript resolve variables?

Part 6 : Hoisting

- What is hoisting?

Hoisting is the process where js moves code at the top.

- Is JavaScript moving code to the top?

yes

- What actually happens during hoisting?

In JS, there are two phases: Memory creation phase and execution context phase.

In the Memory Creation Phase, once js go through all of the code, and creates a memory for all variables and functions.

And in the Execution Context Phase, it initialized with their values where all variables got their values.

eg:

`console.log(x);` //in MCP, js creates memory for x, which waits for initialization.

`let x = 10;` // inECP, js initialize this variable with their value.

`console.log(x)` //here it prints the value.

Here we can say that the js moves code to the top as it creates memory for their first then initialize with the value.

- Which variables are hoisted?

Var – undefined

Let and const – TDZ

- Which functions are hoisted?

Regular functions.

- Are function expressions hoisted? Not
- Are arrow functions hoisted? Not
- Difference between hoisting of `var`, `let`, and `const`.

Var-undefined let and const – TDZ

Part 7: Memory Creation Phase

- What happens during the Memory Creation Phase?

In this phase, js goes through the code and creates memory for the variables and the functions.

- What happens during the Execution Phase?

In this Phase, js goes through the code and initializes variables with their values.

- What values are stored for variables?

For var - undefined

- What values are stored for functions?

Whole function as it is.

Part 8: TDZ

- What is TDZ?

The zone between the declaration and initialization is called Temporal Dead Zone.

- When does TDZ start?

When a variable takes place in the memory.

- When does TDZ end?

When a variable got their value in ECP.

- Why was TDZ introduced?

Because variables have to wait for initialization after once it goes to memory that time before initialization called TDZ, as it is dead before we assign any value.

- Can `typeof` avoid TDZ errors?

Part 9 : Shadowing

- What is variable shadowing?
- Give an example.
- What is illegal shadowing?
- Why is illegal shadowing not allowed?

Part 10: Global Object

- What is the Global Object?

Global object is an object which is created when any variable existed globally in the code.

- Does `var` become a property of `window` in browsers?

yes

- Does `let` become a property of `window`?

not

- Does `const` become a property of `window`?

Not

- What is `globalThis`?

It represents a global object.

- Difference between `window` and `globalThis`.

Window = created when global variables exist and

GlobalThis = used to represent global variables

Part 11: Output based questions

- `console.log(a); var a = 10;`

Undefined

- `console.log(a);`

`let a = 10;`

a can't access before initialization.

- `console.log(a);`

`const a = 10;`

a can't access before initialization.

- `var a = 10;`

`{`

`var a = 20;`

`}`

`console.log(a);`

`20`

- `let a = 10;`

`{`

`let a = 20;`

`}`

`console.log(a);`

`10`

- `let a = 10;`

`{`

`console.log(a);`

`let a = 20;`

`}`

`a` can't access before initialization.

- `var a = 10;`

`function test() {`

`console.log(a);`

`var a = 20;`

`}`

`test();` `//undefined`

- `let a = 10;`
`function test() {`
 `console.log(a);`
`}`
`test();`

10

- `var a = 10;`
`function test() {`
 `console.log(a);`
`}`
`function demo() {`
 `var a = 20;`
 `test();`
`}`
`demo();`

20

- `var a = 1;`
`function foo() {`
 `console.log(a);`
 `var a = 2;`
 `console.log(a);`
`}`
`foo();`

1,2

Part 12 - Error Based question

- `let a = 10;`
`let a = 20;`
Redeclared error
- `const a;`
Initialization error
- `const obj = {`
 `name: "John"`
`};`
`obj.name = "Alex";`
`console.log(obj.name);`
Alex
- `const obj = {};`
`obj = {};`
Constant object doesn't change
- `{`
 `var a = 10;`
`}`
`console.log(a);`
10

Part 13 - Coding question

- Swap two variables **without using a third variable.**
 1. `[a,b] = [b,a];`

1. `a = a+b`

`b = a-b;`

`a=a-b;`

- Write a function to check if a variable is declared.
- Demonstrate block scope using `let`.

```
{  
  let a = 10;  
  
  console.log(a);  
}  
  
console.log(a);
```

- Demonstrate function scope using `var`.

```
var a = 15;  
  
console.log(a);  
  
{  
  var a = 10;  
  
  console.log(a);  
}  
  
console.log(a);
```

- Show an example of variable shadowing.
- Show an example of illegal shadowing.
- Write a program showing TDZ.

```
console.log(a)  
  
let a = 10;
```

- Explain the Memory Creation Phase using code.

```
console.log(a)
```

```
let a = 10;
```

In memory creation phase, js creates a memory space for “a”.

- Show hoisting using a function declaration and a function expression.

```
test();
```

```
function test()
```

```
{
```

```
    console.log("declaration");
```

```
}
```

```
exp();
```

```
const exp = function(){
```

```
    console.log("expression");
```

- Write examples showing declaration, initialization, assignment, redeclaration, and reassignment.

```
var a =10;
```

```
console.log(a)
```

```
var a =20;
```

```
console.log(a)
```

```
a=16;
```

```
console.log(a)
```

```
let b =10;
```

```
console.log(b)
```

```
b =20;

console.log(b)

// let b=16;

// console.log(b)


const c =10;

console.log(c)

// c =20;

// console.log(c)
```

Part 14 - Advanced

- Why was **let** introduced when **var** already existed?

Due to the function scoped and redeclaration of var , as they create bugs.

- Does hoisting happen at compile time or runtime?

Compile time

- Does JavaScript parse the whole file before execution?

Yes

- Why does **let** stay in the TDZ even though it's hoisted?

Because they to get initialized with their value

- How does the JavaScript engine implement hoisting internally?

In JS, there are two phases: Memory creation phase and execution context phase.

In the Memory Creation Phase, once js go through all of the code, and creates a memory for all variables and functions.

And in the Execution Context Phase, it initialized with their values where all variables got their values.

eg:

`console.log(x);` //in MCP, js creates memory for x, which waits for initialization.

`let x = 10;` // inECP, js initialize this variable with their value.

`console.log(x)` //here it prints the value.

Here we can say that the js moves code to the top as it creates memory for their first then initialize with the value.

- Why are function declarations fully hoisted but function expressions are not?

Because function expressions are treated as the variables

- Is hoisting different in modules than in classic scripts?

no

- Why does `typeof undeclaredVariable` work, but `typeof` on a TDZ variable throws?
- Explain how lexical scope and closures are related.

Lexical scope is the scope of outer environment and in closures, the variables accessed from lexical scope after their function returned

Eg:

```
function x(){  
  let a = 10;  
  function y(){  
    console.log(a);  
  }  
  y();  
}
```

`x();`

- If you were designing JavaScript today, would you keep `var`? Why or why not?

No , because it creates bugs in large codebase due to function scoped and redeclaration.

